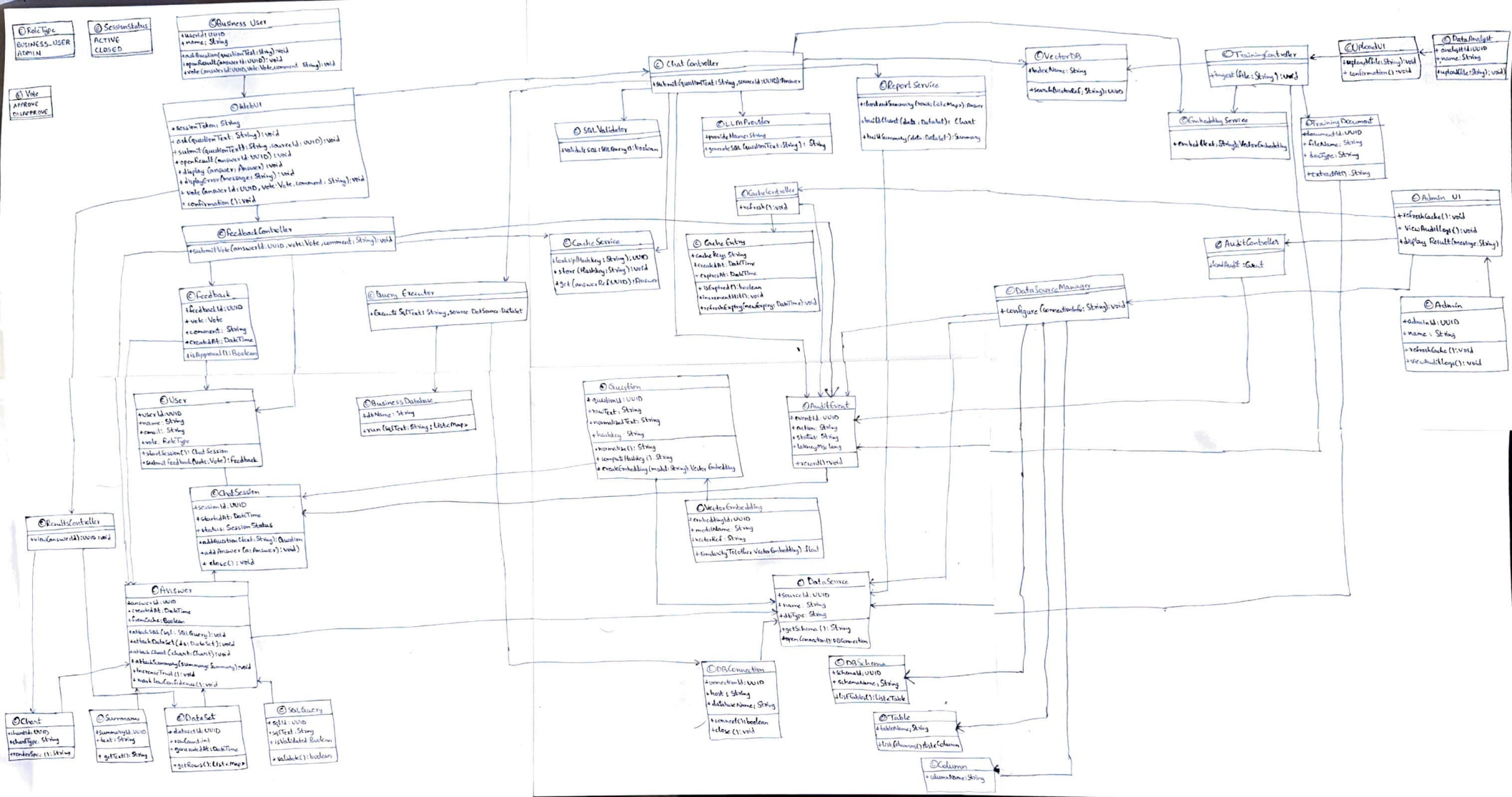
RoleType
BUSINESS_USER
ADMIN
SessionStatus
ACTIVE
CLOSED
Vote
APPROVE
DISAPPROVE
BusinessUser
+userId: UUID
+name: String
+addQuestion(questionText: String): void
+openResult(answerId: UUID): void
+vote(answerId: UUID, vote: Vote, comment: String): void
WebUI
+sessionToken: String
+ask(questionText: String): void
+submit(questionTextId: String, sourceId: UUID): void
+openResult(answerId: UUID): void
+display(answer: Answer): void
+displayError(message: String): void
+vote(answerId: UUID, vote: Vote, comment: String): void
+confirmation(): void
FeedbackController
+submitVote(answerId: UUID, vote: Vote, comment: String): void
Feedback
+feedbackId: UUID
+vote: Vote
+comment: String
+createdAt: DateTime
+isApproval(): Boolean
User
+userId: UUID
+name: String
+email: String
+role: RoleType
+startSession(): ChatSession
+submitFeedback(note: Vote): Feedback
ChatSession
+sessionId: UUID
+startedAt: DateTime
+status: SessionStatus
+addQuestion(text: String): Question
+addAnswer(a: Answer): void
+close(): void
ResultsController
+view(answerId): UUID: void
Answer
+answerId: UUID
+createdAt: DateTime
+fromCache: Boolean
+attachSQL(sql: SQLQuery): void
+attachDataSet(ds: DataSet): void
+attachChart(chart: Chart): void
+attachSummary(summary: Summary): void
+increaseTrust(): void
+markLowConfidence(): void
Chart
+chartId: UUID
+chartType: String
+renderSpec(): String
Summary
+summaryId: UUID
+text: String
+getText(): String
DataSet
+datasetId: UUID
+rowCount: int
+generatedAt: DateTime
+getRows(): List<Map>
SQLQuery
+qId: UUID
+qlText: String
+isValidated: Boolean
+isValid(): boolean
BusinessDatabase
+dbName: String
+runQ(qText: String): List<Map>
QueryExecutor
+Execute(qlText: String, source: DataSource): DataSet
ChatController
+submit(questionText: String, sourceId: UUID): Answer
SQLValidator
+validate(sql: SQLQuery): boolean
LLMProvider
+providerName: String
+generateSQL(questionText: String): String
CacheService
+lookUp(Hashkey: String): UUID
+store(Hashkey: String): void
+get(answerId: UUID): Answer
CacheEntry
+cacheKey: String
+createdAt: DateTime
+expiresAt: DateTime
+isExpired(): boolean
+incrementHit(): void
+refreshExpiry(newExpiry: DateTime): void
CacheController
+refresh(): void
Question
+questionId: UUID
+rawText: String
+normalizedText: String
+hashkey: String
+normalize(): String
+computeHashkey(): String
+createEmbedding(modelId: String): VectorEmbedding
ReportService
+buildAndSummary(rows: List<Map>): Answer
+buildChart(data: DataSet): Chart
+buildSummary(data: DataSet): Summary
VectorDB
+indexName: String
+search(vectorRef: String): UUID
TrainingController
+ingest(file: String): void
UploadUI
+uploadFile(file: String): void
+confirmation(): void
DataAnalyst
+analystId: UUID
+name: String
+uploadFile(file: String): void
EmbeddingService
+embed(text: String): VectorEmbedding
TrainingDocument
+documentId: UUID
+fileName: String
+docType: String
+extract(path): String
AuditEvent
+eventId: UUID
+action: String
+status: String
+latencyMs: long
+record(): void
AuditController
+listAudit: Count
AdminUI
+refreshCache(): void
+viewAuditLogs(): void
+displayResult(message: String)
Admin
+adminId: UUID
+name: String
+refreshCache(): void
+viewAuditLogs(): void
DataSourceManager
+configure(connectionInfo: String): void
VectorEmbedding
+embeddingId: UUID
+modelName: String
+vectorRef: String
+similarityTo(other: VectorEmbedding): float
DataSource
+sourceId: UUID
+name: String
+dbType: String
+getSchema(): String
+openConnection(): DBConnection
DBConnection
+connectionId: UUID
+host: String
+databaseName: String
+connect(): boolean
+close(): void
DBSchema
+schemaId: UUID
+schemaName: String
+listTables(): List<Table>
Table
+tableName: String
+list(columns): List<Column>
Column
+columnName: String